SISTEMA BÚSSÓLA

Plataforma de Administração e Controle para Clubes de Desbravadores

DOCUMENTO DE ARQUITETURA DE SOFTWARE

Versão 2.0.0 — Abril de 2026

Objetivo deste Documento

Este documento descreve as principais decisões de projeto tomadas durante o desenvolvimento do Sistema Bússóla, os critérios considerados durante a tomada dessas decisões, e as visões da arquitetura do sistema. O documento adota uma estrutura baseada no modelo de visão "4+1" [KRU41].

Histórico de Revisão

Data	Versão	Demanda	Descrição	Autor
Jun/2025	1.0.0	Especialidade AP-052	Versão inicial do projeto	Luiz Phillipe de Souza Santos
Abr/2026	2.0.0	Especialidade AP-052	Reformulação completa: nova UI, documentação formal, arquitetura revisada	Luiz Phillipe de Souza Santos

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do Sistema Bússóla, usando diversas visões de arquitetura para representar diferentes aspectos do sistema. O objetivo é capturar e comunicar as decisões arquiteturais significativas, orientando o desenvolvimento, manutenção e evolução do produto.

O documento adota uma estrutura baseada no modelo de visão "4+1" [KRU41], adaptado para o contexto de uma aplicação web com BaaS.

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao Sistema Bússóla, desenvolvido como projeto para a Especialidade de Informática Programável (AP-052). O sistema abrange:

- Autenticação segura via Supabase Auth (email/senha + Google OAuth).
- Cadastro e gestão de unidades, desbravadores, classes e especialidades.
- Controle financeiro: caixa com entradas, saídas e custos; mensalidades por membro.
- Gestão de patrimônio e inventário de bens do clube.
- Registro de atas de reunião e atos administrativos com suporte a anexos.
- Geração de 6 tipos de relatórios em PDF via jsPDF executado no cliente.

1.3 Definições, Acrônimos e Abreviações

Abreviação / Termo	Definição
BaaS	Backend as a Service – modelo de backend gerenciado, sem servidor próprio.
JWT	JSON Web Token – padrão de autenticação stateless via token assinado (RFC 7519).
RLS	Row Level Security – políticas de acesso por linha no PostgreSQL.
PostgREST	API REST gerada automaticamente a partir do esquema PostgreSQL pelo Supabase.
SPA	Single Page Application – não aplicado diretamente; o sistema usa páginas HTML tradicionais.
OAuth 2.0	Protocolo de autorização usado para login com Google.
jsPDF	Biblioteca JavaScript client-side para geração de documentos PDF no navegador.
GitHub Pages	Serviço gratuito de hospedagem de sites estáticos a partir de repositórios GitHub.
4+1	Modelo de arquitetura de Philippe Kruchten que organiza o sistema em 5 visões: Casos de Uso, Lógica, Implementação, Implantação e Processo.

1.4 Referências

- [KRU41]: The "4+1" view model of software architecture – Philippe Kruchten, IEEE Software, November 1995.

- Supabase Documentation. https://supabase.com/docs
- Documento de Visão – Sistema Bússóla, v2.0.0, Abr/2026.
- Especificação de Casos de Uso – Sistema Bússóla, v2.0.0, Abr/2026.
- Especificação de Requisitos de Software (SRS) – Sistema Bússóla, v2.0.0, Abr/2026.
- Repositório público no GitHub (código-fonte do sistema).

2. REPRESENTAÇÃO ARQUITETURAL

Este documento detalha as visões baseadas no modelo "4+1" [KRU41]. As visões utilizadas são:

Visão	Público-Alvo	Área de Interesse
Casos de Uso	Todos os stakeholders	Requisitos funcionais e comportamento do sistema.
Lógica	Analistas e Arquitetos	Organização em camadas, módulos e fluxos.
Implementação	Programadores	Estrutura de arquivos, componentes e dados.
Implantação	Administrador / DevOps	Hospedagem, serviços externos e infraestrutura.
Processo	Analistas	Performance, segurança e disponibilidade.

2.1 Diagrama de Relações (Visão Geral)

A arquitetura do Sistema Bússóla adota o estilo em Camadas (Three-Tier / N-Tier), com frontend estático e backend gerenciado desacoplados, comunicando-se via API REST HTTPS. A representação a seguir sintetiza os componentes principais:

[CAMADA DE APRESENTAÇÃO]
HTML5 + CSS3 + JavaScript ES2022+ | Páginas estáticas no navegador
↕ **HTTPS (REST/JSON) via @supabase/supabase-js SDK** ↕
[CAMADA DE BACKEND (Supabase BaaS)]
Supabase Auth (JWT + Google OAuth) | PostgREST (API REST) | Supabase Storage
↕ **SQL / JDBC** ↕
[CAMADA DE PERSISTÊNCIA]
PostgreSQL 15 | Row Level Security (RLS) | Tabelas: unidade, desbravador, mensalidade, caixa, patrimônio, atas, atos
[HOSPEDAGEM: GitHub Pages (Frontend) | Supabase Cloud (Backend)]

Figura 1 – Diagrama de Relações da Arquitetura do Sistema Bússola

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e as restrições que têm impacto significativo na arquitetura. (Ref.: Seções 6 e 7 do Documento de Visão.)

Dimensão	Detalhamento
Linguagem	Frontend: JavaScript ES2022+ com HTML5 e CSS3. Sem framework JS (React/Vue/Angular).
Backend	Supabase BaaS: PostgreSQL 15, Supabase Auth, Supabase Storage, PostgREST.
Hospedagem	Frontend: GitHub Pages ou Vercel (gratuito). Backend: Supabase Cloud (Free Tier).
Segurança	Autenticação JWT gerenciada pelo Supabase Auth. HTTPS obrigatório. RLS ativo em todas as tabelas. Senhas com mínimo de 6 caracteres.
Geração de PDF	Biblioteca jsPDF v2.5.1 executada client-side, eliminando necessidade de servidor.
Escalabilidade	Supabase Free Tier suporta até 500MB de banco e 50.000 usuários; suficiente para o porte do projeto.
Manutenibilidade	Cada módulo em arquivo separado (HTML + JS). Código em português. Documentação completa em 4 documentos.
Portabilidade	Interface responsiva compatível com Chrome 110+, Firefox 110+, Safari 16+, Edge 110+.
Desempenho	Carregamento de página < 3s em 4G. Geração de PDF < 5s. Dependente da latência do Supabase.

Tabela 1 – Requisitos e restrições arquiteturais

4. VISÃO DE CASOS DE USO

Esta seção lista os casos de uso centrais e significantes para a arquitetura do Sistema Bússóla. A especificação completa está no documento "Especificação de Casos de Uso – Sistema Bússóla, v2.0.0."

Lista dos casos de uso do sistema:

- UC-001 – Realizar Login com Email/Senha
- UC-002 – Realizar Login com Google OAuth
- UC-003 – Realizar Cadastro de Conta
- UC-004 – Realizar Logout
- UC-005 – Cadastrar Unidade
- UC-006 – Editar ou Excluir Unidade
- UC-007 – Cadastrar Desbravador
- UC-008 – Editar ou Excluir Desbravador
- UC-009 – Registrar Mensalidade
- UC-010 – Atualizar Status de Mensalidade
- UC-011 – Registrar Movimentação de Caixa
- UC-012 – Cadastrar / Editar / Excluir Item de Patrimônio
- UC-013 – Registrar Ata ou Ato
- UC-014 a UC-016 – Gerar Relatórios em PDF (6 tipos)

4.1 Casos de Uso Significantes para a Arquitetura

Caso de Uso	Ator(es)	Impacto Arquitetural
UC-001 – Login Email/Senha	Todos os usuários	Define a camada de segurança: Supabase Auth, JWT automático, proteção de rotas via getSession().
UC-002 – Login Google OAuth	Todos os usuários	Define integração OAuth 2.0; requer configuração de redirect URL no Supabase.
UC-005/007 – Cadastros	Secretário, Conselheiro	Define o esquema relacional central: unidade, desbravador, classe, especialidades, tabela pivot.
UC-009/011 – Financeiro	Tesoureiro	Define tabelas mensalidade e caixa; cálculos client-side de totais e saldo.
UC-013 – Atas e Atos	Secretário	Define uso de Supabase Storage para upload de anexos e URL pública.
UC-014–016 – Relatórios PDF	Diretor, Secretário, Tesoureiro	Define arquitetura client-side da geração de documentos: jsPDF sem servidor.

Tabela 2 – Casos de uso significantes para a arquitetura

5. VISÃO LÓGICA

5.1 Visão Geral – Pacotes e Camadas

Camada	Módulo / Arquivo	Responsabilidade
Apresentação	index.html, dashboard.html, unidade.html, desbravador.html, mensalidade.html, caixa.html, patrimonio.html, atas_atos.html, relatorios.html	UI, navegação entre páginas, coleta de entrada do usuário e exibição de dados.
Lógica de Negócio (cliente)	app.js, unidade.js, desbravador.js, mensalidade.js, atas_atos.js	Validação de campos, chamadas ao Supabase, montagem dinâmica de tabelas, cálculo de totais.
Geração de Documentos	relatorios/relatorio_*.html	Busca de dados do Supabase e geração de PDFs via jsPDF client-side.
Configuração / Autenticação	supabase.js, verificarSessao.js	Inicialização do cliente Supabase; proteção de rotas com verificação de sessão JWT.
Estilo Global	style.css	Design system: variáveis de cor, tipografia, layout sidebar, cards, tabelas, modais, badges.
Persistência (Backend)	Supabase: PostgreSQL + Auth + Storage + PostgREST	Armazenamento de dados, autenticação, arquivos e API REST automática.

Tabela 3 – Visão geral de camadas e módulos

5.2 Estrutura de Arquivos

Arquivo	Responsabilidade
index.html	Tela de login com email/senha e Google OAuth.
cadastro.html	Cadastro de novos usuários.
dashboard.html	Painel de controle com estatísticas e acesso rápido.
unidade.html + unidade.js	CRUD completo de unidades.
desbravador.html + desbravador.js	CRUD completo de desbravadores com especialidades.
mensalidade.html + mensalidade.js	Registro e gestão de mensalidades.
caixa.html	Registro de movimentações financeiras (entradas/saídas/custos).
patrimonio.html	Inventário de bens do clube.
atas_atos.html + atas_atos.js	Registro de atas e atos com abas separadas.
relatorios.html	Hub de acesso aos 6 relatórios.
relatorios/relatorio_saida.html	PDF de Autorização de Saída por unidade.

relatorios/relatorio_fluxo_caixa.html	PDF de Fluxo de Caixa com filtro de período.
relatorios/relatorio_patrimonio.html	PDF de Inventário de Patrimônio.
relatorios/relatorio_atas_atos.html	PDF do Livro de Atas e Atos.
relatorios/relatorio_mensalidades.html	PDF de Mensalidades por ano.
relatorios/relatorio_geral.html	PDF do Relatório Geral do Clube.
style.css	Design system global.
supabase.js	Inicialização do cliente Supabase (URL + chave anon).
app.js	Utilitários compartilhados: toast, modal, formatação, sidebar, ícones SVG.

6. VISÃO DE IMPLEMENTAÇÃO

6.1 Diagrama de Sequência – Login (UC-001)

Diagrama de Sequência – UC-001 (Login)

Cliente (Navegador) → informar { email, senha } → index.html
index.html → validar campos → JavaScript
JavaScript → supabase.auth.signInWithPassword({ email, password }) → Supabase Auth
Supabase Auth → verificar hash da senha no PostgreSQL → PostgreSQL
Supabase Auth ← resultado (ok / erro) ← PostgreSQL
Supabase Auth → emitir { accessToken JWT, session } → JavaScript
JavaScript → armazenar token no LocalStorage → Navegador
Cliente ← redirect para dashboard.html ← JavaScript

Figura 2 – Diagrama de Sequência UC-001 (Login)

6.2 Diagrama de Sequência – Gerar PDF (UC-014 a UC-016)

Diagrama de Sequência – UC-014 a UC-016 (Geração de PDF)

Usuário → selecionar filtros (unidade / período / ano) → Relatório HTML
Usuário → clicar em "Baixar PDF" → Relatório HTML
JavaScript → supabase.from(tabela).select(*) → Supabase PostgREST
Supabase PostgREST → consultar PostgreSQL → PostgreSQL
JavaScript ← dados JSON ← Supabase PostgREST
JavaScript → new jsPDF() → jsPDF (client-side)
jsPDF → doc.text(), doc.line(), doc.rect() → Buffer PDF em memória
jsPDF → doc.save("arquivo.pdf") → Download no navegador
Usuário ← arquivo PDF salvo no dispositivo ← Navegador

Figura 3 – Diagrama de Sequência – Geração de PDF (UC-014 a UC-016)

6.3 Diagrama Entidade-Relacionamento (DER)

Entidade	Atributos Principais	Chave	Relacionamento
conselheiro	id, nome, telefone, email	PK: id	→ 1:N unidade
unidade	id, nome, conselheiro_id, associado1_id, associado2_id	PK: id FK: conselheiro_id	→ 1:N desbravador
desbravador	id, nome, unidade_id, classe_id, data_nascimento, contato	PK: id FK: unidade_id, classe_id	→ N:N especialidades
classe	id, nome, descricao	PK: id	→ 1:N desbravador
especialidades	id, nome, descricao	PK: id	← N:N desbravador
desbravador_especialidade	id, desbravador_id, especialidade_id, concluida	PK: id FK: ambos	Tabela pivot
mensalidade	id, desbravador_id, mes, ano, valor, status, anexo_url	PK: id FK: desbravador_id	N:1 desbravador
caixa	id, tipo, descricao, valor, data	PK: id	Independente
patrimonio	id, item, quantidade, estado, unidade_id, observacoes	PK: id FK: unidade_id	N:1 unidade
atas	id, titulo, conteudo, data, anexo_url	PK: id	Independente
atos	id, titulo, conteudo, data, anexo_url	PK: id	Independente
usuarios	id, auth_user_id, nome, perfil_id, unidade_id	PK: id	Vinculado ao Supabase Auth
perfis	id, nome	PK: id	→ 1:N usuarios

Tabela 4 – Diagrama Entidade-Relacionamento (DER) do Sistema Bússola

7. VISÃO DE IMPLANTAÇÃO

7.1 Infraestrutura

Nodo / Serviço	Descrição	Hospedagem
Frontend (HTML/CSS/JS)	Arquivos estáticos servidos diretamente ao navegador do usuário.	GitHub Pages ou Vercel (gratuito)
Supabase Auth	Serviço de autenticação JWT com suporte a e-mail/senha e Google OAuth.	Supabase Cloud
PostgreSQL 15	Banco de dados relacional com RLS e PostgREST.	Supabase Cloud
Supabase Storage	Armazenamento de arquivos (comprovantes de mensalidades, anexos de atas).	Supabase Cloud
jsPDF (client)	Geração de PDFs executada no navegador do usuário; sem servidor adicional.	Client-side (navegador)

8. DECISÕES ARQUITETURAIS

8.1 Por que BaaS (Supabase) em vez de backend próprio?

- Reduz drasticamente a complexidade de infraestrutura; não requer gerenciamento de servidor.
- Supabase oferece Free Tier generoso (500MB de banco, 1GB storage, 50k usuários).
- PostgreSQL é robusto, suporta JOINs complexos e RLS nativo.
- Supabase Auth resolve autenticação segura (JWT + OAuth) sem código adicional.
- O SDK JavaScript (@supabase/supabase-js) é bem documentado e alinhado à linguagem do projeto.

8.2 Por que HTML/CSS/JS puro em vez de React/Vue?

- Alinhado ao escopo educacional da especialidade: não requer NPM, bundler ou processo de build.
- Pode ser aberto diretamente em qualquer navegador sem configuração.
- Mais acessível para compreensão e apresentação ao examinador.
- Menor curva de aprendizado para evolução futura pelo responsável do clube.

8.3 Por que geração de PDF no cliente?

- Não requer servidor adicional para geração de documentos.
- Funciona após o carregamento da página mesmo sem conexão contínua.
- A biblioteca jsPDF é madura, amplamente utilizada e bem documentada.
- Elimina custo de servidor de PDF serverless (ex: Puppeteer em cloud).

8.4 Padrao de Código Adotado

- Separação de responsabilidades: HTML para estrutura, CSS para estilo, JS para comportamento.
- async/await para todas as operações assíncronas com o Supabase.
- Toast notifications (app.js) para feedback visual em todas as ações.
- Validação de campos antes de qualquer operação de escrita no banco.
- Verificação de sessão JWT no carregamento de cada página protegida.
- Padrão de modal: confirmação antes de qualquer exclusão de dado.